

Lab 5 — Your First Device Driver: I²C and the IMU Bring-Up

ESP32-S3 FIRMWARE LAB SERIES · PHASE 1 · FOUNDATIONS

Estimated time	3–4 hrs
Introduces	bus addressing, register maps, signed reassembly
Prerequisites	Labs 0–4.
Beyond the board	An MPU-6500 / MPU-6050-class IMU breakout and four jumper wires.
Target	ESP32-S3 (written against an N16R8 module: 16 MB flash, 8 MB octal PSRAM)
Framework	ESP-IDF v5.5, CMake and Ninja, driven by <code>idf.py</code>

NOTE

How to read this lab

Sections run §5.1 objective, §5.2 parts and wiring, §5.3 how it works, then the code, a line-by-line walkthrough, the documentation map, and verification. Callout boxes are tagged **[WARNING]**, **[KEY POINT]**, **[NOTE]**, **[DOCS]** and **[TM4C BRIDGE]**. Figures are given as captioned text and tables so every value in them is searchable and checkable.

The **TM4C bridge** notes map each topic onto the equivalent bare-metal, register-level work on a TI TM4C123 (Cortex-M4F). Skip them if that is not your background; nothing later depends on them.

5.1 Objective & TM4C bridge

Bring up a real sensor over a real bus: address it, confirm its identity, wake it, and convert its raw output into physical units you can sanity-check against gravity. This is the first lab where you are talking to somebody else's silicon, which means the datasheet stops being optional.

TM4C BRIDGE

Most bare-metal courses cover I²C as theory only. For many people this is the first time they drive it, and more importantly the first time you work from a *manufacturer's register map* rather than the MCU's own datasheet. That skill — open an unfamiliar chip's documentation and turn it into working code — is the one that transfers to every part you will ever use, and it is the real subject of this lab.

WARNING

Your part returns 0x70, not 0x68

Common GY-9250 / GY-6500 style breakouts carry an **MPU-6500**, whose WHO_AM_I returns **0x70**. A true MPU-6050 returns **0x68**; the MPU-9250 returns **0x71**. All of them answer at bus address **0x68**, which is a completely separate number — §5.3 explains why. **Reading 0x70 means your wiring is right.** Every register this lab touches is identical across these parts, so nothing else changes.

5.2 Parts & wiring

Item	Status	Notes
IMU breakout (MPU-6500 or GY-521/MPU-6050)	Required	Not onboard. Nothing on the ESP32-S3 board is an IMU. If you cannot find it, it is a \$5 part sold as "GY-521 MPU6050."
4 jumper wires	Required	Female-to-female direct, or male-to-male plus a breadboard.
Logic analyzer	strongly recommended	This is the best protocol capture in the whole track.

Sensor pin	ESP32-S3	Notes
VCC	3V3	Not 5V. The logic must stay at 3.3 V.
GND	GND	Connect first.
SDA	GPIO8	Serial data
SCL	GPIO9	Serial clock
XDA, XCL, AD0, INT	—	Leave unconnected.

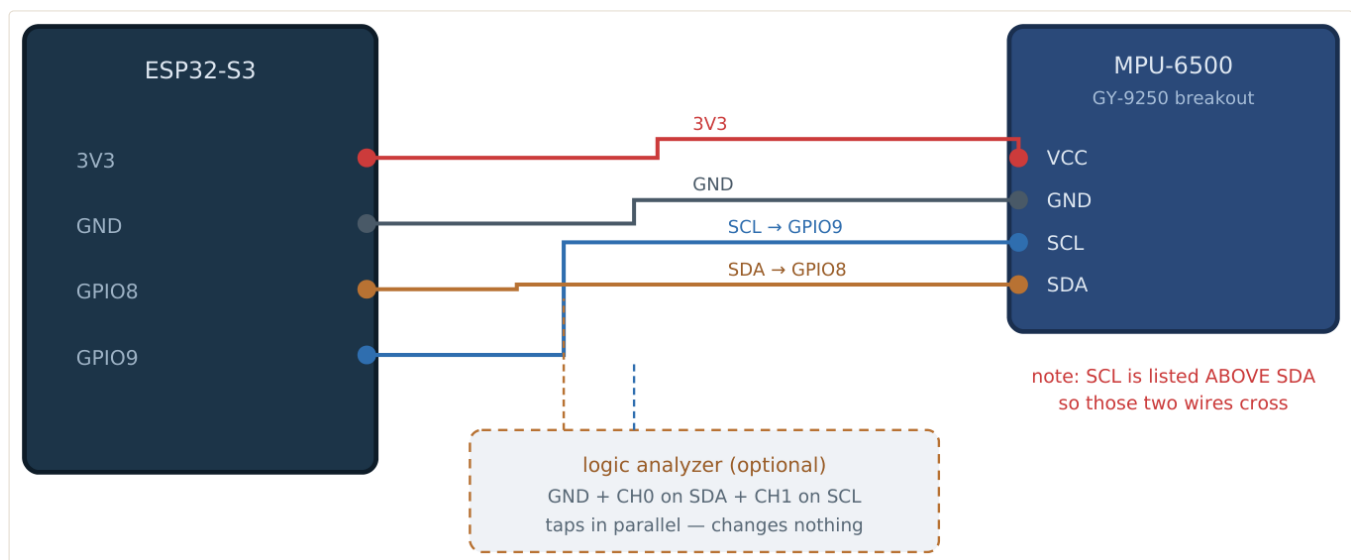


Figure 5.1. IMU wiring. The SDA and SCL wires cross, and that is correct — the breakout's header lists SCL above SDA. The analyzer taps the same two lines in parallel and changes nothing about the circuit.

IMU wiring, with the logic analyzer tapped in parallel.

Sensor pin	ESP32-S3	Note
VCC	3V3	Not 5V
GND	GND	Connect first

Sensor pin	ESP32-S3	Note
SDA	GPIO8	Serial data
SCL	GPIO9	Serial clock
XDA, XCL, AD0, INT	not connected	leave unwired

The SDA and SCL wires physically CROSS, and that is correct: on the GY-521 / GY-9250 header, SCL sits ABOVE SDA, the opposite of the order you would guess. Wire by label, never by header position.

The logic analyzer taps the same two lines in parallel and changes nothing electrically: CH0 to SDA, CH1 to SCL, plus a ground wire.

WARNING

The mistake that looks exactly like a dead sensor

On the GY-521/GY-9250 header, **SCL sits above SDA** — the opposite of the order you would guess. So those two wires physically cross. **Wire by label, never by header position.** Swapping them produces precisely the symptom of a broken part: WHO_AM_I reads `0x00`, or the transaction is not acknowledged at all.

WARNING

Two common modules look alike and both live at 0x68

The **GY-521** is the IMU. The **DS1307 / Tiny RTC** is a real-time clock and measures no motion — and by genuine coincidence it shares address `0x68`, so scanning the bus will not distinguish them. Fastest tells: the RTC has a **round coin-cell holder**; the IMU does not. And the IMU's header ends in **XDA XCL AD0 INT**, labels unique to it.

NOTE

Analyzer hookup

GND first, then CH0 to SDA and CH1 to SCL. Sample at **4 MHz** with 1 M samples, rename the channels before adding the **I²C decoder**, and trigger on SDA falling. You are tapping in parallel, so the sensor stays wired exactly as it is — put each signal in its own breadboard row, so the sensor wire, the board wire and the probe wire all share one electrical node.

5.3 How it works

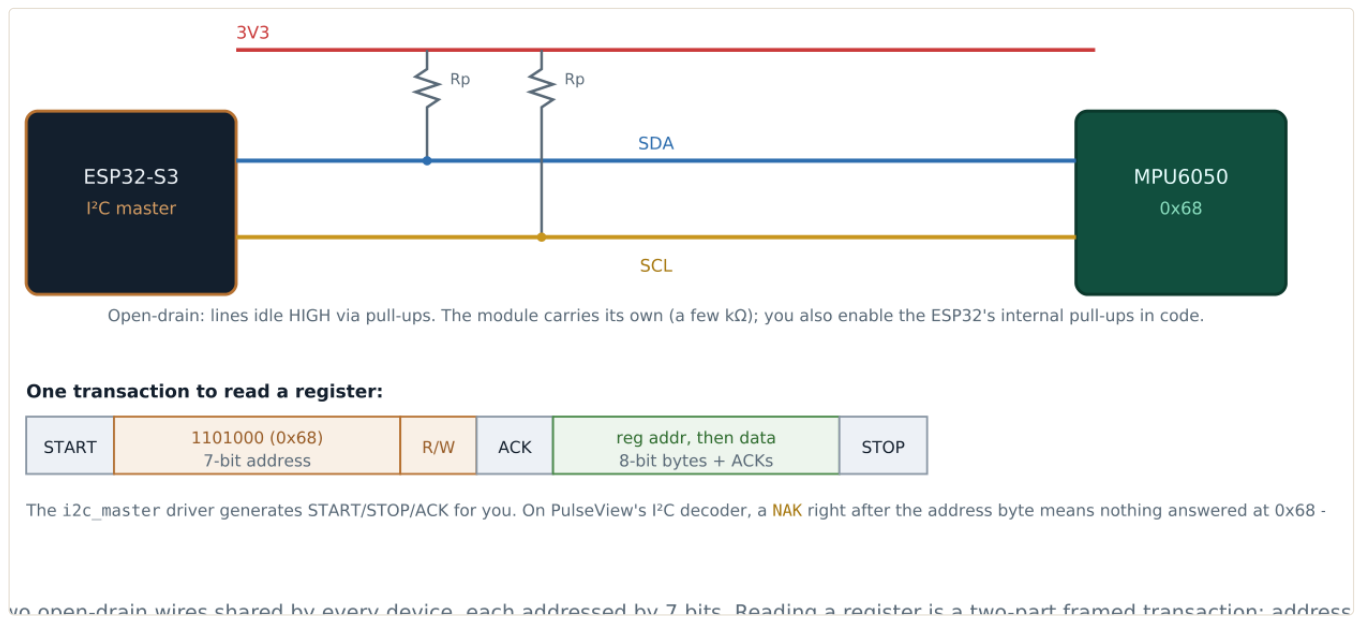


Figure 5.2. I²C is two open-drain wires shared by every device, each addressed by 7 bits. Reading a register is a two-part framed transaction: address the chip, say which register, then read the bytes. This is the picture to have in your head when you put the analyzer on the bus.

I2C bus topology and one register-read transaction.

Topology: ESP32-S3 (I²C master) and MPU6050 (address 0x68) both connect to the shared SDA and SCL lines. A 3V3 rail feeds two pull-up resistors (Rp), one to SDA and one to SCL.

Open-drain: the lines idle HIGH via the pull-ups. Devices can only pull a line LOW, never drive it high. The breakout module carries its own pull-ups (a few kilohms); the code also enables the ESP32's internal ones.

One transaction to read a register, in order:

Field	Contents
START	bus start condition
Address	1101000 binary = 0x68, 7-bit address
R/W	direction bit
ACK	from the addressed device
Data	register address, then the data bytes, each followed by an ACK
STOP	bus stop condition

The `i2c_master` driver generates START, STOP and ACK for you. On PulseView's I²C decoder, a NAK immediately after the address byte means nothing answered at 0x68.

Two wires, many devices

I²C solves a different problem from UART. UART is point-to-point: one link, two devices, and a separate pair of wires for each new device. I²C is a **bus**: every device hangs off the same two wires, SDA (data) and SCL (clock), and each one has an **address** so the controller can pick which to talk to. Add a tenth sensor and you add zero wires.

The wires are **open-drain**, which is worth understanding rather than memorising. Each device can only pull a line *low*; none can drive it high. A **pull-up resistor** returns the line to high when nobody is pulling. That arrangement means two devices talking at once can never short a driving-high output against a driving-low one — the worst case is simply that the line goes low. It is what makes a shared bus electrically safe, and it is why pull-ups are mandatory rather than optional. Your breakout carries its own, and the lab also enables the ESP32's internal ones; both together just lowers the effective resistance slightly, which is a non-issue at these speeds and distances.

Bus address versus register address

These are two unrelated numbers that look confusingly similar, and confusing them is the single most common source of I²C bewilderment.

	Bus address	Register address
Names	Which chip on the shared wires	Which byte inside that chip
Set by	The I ² C standard and the manufacturer; sometimes selectable via the AD0 pin	The manufacturer, in the register map document
Width	7 bits, so 128 possible addresses (0x00–0x7F)	8 bits, so 256 possible registers (0x00–0xFF)
Yours	<code>0x68</code>	<code>0x75</code> WHO_AM_I, <code>0x6B</code> PWR_MGMT_1, <code>0x3B</code> ACCEL_XOUT_H, <code>0x1C</code> ACCEL_CONFIG
In code	Set once, in <code>i2c_device_config_t</code>	Passed on every read and write

So `0x68` and `0x6B` are not related despite looking almost identical, and neither has anything to do with the ESP32's own memory map. The value *stored inside* WHO_AM_I — `0x70` on your part — is a third thing again: it is data, not an address. When the analyzer decodes your capture it shows the address as `0x68` and the data byte as `0x70`, and both are simultaneously correct.

Why 7 bits gives 128: each bit doubles the count, so $2^7 = 128$. Equivalently, a 7-bit field splits as $2^3 \times 2^4 = 8 \times 16$, which is why a 7-bit address fits in two hex digits with the top digit limited to 0–7.

The shape of a register read

Reading a register is two transactions joined by a **repeated start**. First the controller sends START, the 7-bit address with the write bit, and the register number — that tells the chip "here is where I want to read from." Then, instead of releasing the bus, it issues another START, re-sends the address with the read bit, and clocks out the data bytes. Holding the bus across both halves is what prevents another controller from interleaving and moving the register pointer underneath you. The driver's `i2c_master_transmit_receive` does exactly this in one call.

Where the numbers come from

Nothing in this lab's code is arbitrary, and this is the moment to internalise that. `0x75`, `0x6B`, `0x3B` and the divisor `16384` all come out of InvenSense's documentation for your part — the register map for the addresses, and the product specification's sensitivity table for the 16384 counts per g at the default $\pm 2\text{g}$ range. Trace every one of them back to its source table before you write it into code. Download both PDFs before starting this lab and keep them next to the project folder.

5.4 Procedure & code

1. Board unplugged. Wire GND first, then VCC, then SDA and SCL by label.
2. Create the project, set the target, type the listing.
3. Flash and watch WHO_AM_I. Do not proceed until that value is right.

```
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/i2c_master.h"
#include "esp_log.h"

#define SDA_PIN    8
#define SCL_PIN    9
#define MPU_ADDR   0x68

static const char *TAG = "lab5";
static i2c_master_dev_handle_t mpu;

static esp_err_t mpu_write(uint8_t reg, uint8_t val) {
    uint8_t buf[2] = { reg, val };
    return i2c_master_transmit(mpu, buf, 2, 1000);
}

static esp_err_t mpu_read(uint8_t reg, uint8_t *dst, size_t n) {
    return i2c_master_transmit_receive(mpu, &reg, 1, dst, n, 1000); // write reg addr, then read n
}

void app_main(void) {
    i2c_master_bus_config_t bus_cfg = {
        .clk_source      = I2C_CLK_SRC_DEFAULT,
        .i2c_port         = I2C_NUM_0,
        .sda_io_num       = SDA_PIN,
        .scl_io_num       = SCL_PIN,
        .glitch_ignore_cnt = 7,
        .flags.enable_internal_pullup = true, // the breakout also has its own pull-ups
    };
    i2c_master_bus_handle_t bus;
    ESP_ERROR_CHECK(i2c_new_master_bus(&bus_cfg, &bus));

    i2c_device_config_t dev_cfg = {
        .dev_addr_length = I2C_ADDR_BIT_LEN_7,
        .device_address   = MPU_ADDR,
        .scl_speed_hz     = 400000, // 400 kHz fast mode
    };
    ESP_ERROR_CHECK(i2c_master_bus_add_device(bus, &dev_cfg, &mpu));

    uint8_t who = 0;
    mpu_read(0x75, &who, 1); // WHO_AM_I
```

```

ESP_LOGI(TAG, "WHO_AM_I = 0x%02X (0x70 = MPU-6500, 0x68 = MPU-6050, 0x71 = MPU-9250)", who);

mpu_write(0x6B, 0x00); // PWR_MGMT_1 = 0 -> wake from sleep
vTaskDelay(pdMS_TO_TICKS(50));

while (1) {
    uint8_t r[6];
    mpu_read(0x3B, r, 6); // ACCEL_XOUT_H .. ACCEL_ZOUT_L, 6 bytes
    int16_t ax = (int16_t)((r[0] << 8) | r[1]); // each axis is a big-endian 16-bit word
    int16_t ay = (int16_t)((r[2] << 8) | r[3]);
    int16_t az = (int16_t)((r[4] << 8) | r[5]);
    // default full scale is +/-2g -> 16384 LSB per g
    ESP_LOGI(TAG, "ax=%+.2f ay=%+.2f az=%+.2f (g)",
              ax / 16384.0f, ay / 16384.0f, az / 16384.0f);
    vTaskDelay(pdMS_TO_TICKS(200));
}
}

```

5.5 Code walkthrough, line by line

Fifty-eight lines, and the densest listing so far because it is your first real device driver. Read it slowly; several lines here are the ones interviewers pick on.

Line 3 · `#include "driver/i2c_master.h"`

ESP-IDF's **modern** I²C master driver. There is an older API in `driver/i2c.h` which is now deprecated, and a great deal of code on the internet still uses it — if an example will not compile against your v5.5 install, this is very often why.

Line 8 · `#define MPU_ADDR 0x68`

The sensor's **bus address** — which chip on the shared wires, not which register inside it. Keep the §5.3 table in mind: this number and `0x6B` on line 44 look almost identical and are completely unrelated.

Line 11 · `static i2c_master_dev_handle_t mpu;`

The device handle, file-scope so both helpers can reach it. Same opaque-token idea as Lab 1's `strip`: you never look inside it, you pass it to the driver so it knows which device you mean.

Lines 13-16 · `mpu_write(uint8_t reg, uint8_t val)`

Wraps a raw driver call into a readable operation. `uint8_t buf[2] = { reg, val };` packs the pair into a two-byte buffer, because the I²C convention is **first byte says which register, second byte is the data**. Then `i2c_master_transmit(mpu, buf, 2, 1000)` sends both bytes with a 1000 ms timeout.

The return type `esp_err_t` is passed straight back so a caller *could* check it. Note that `app_main` never does — a real weakness, and one worth fixing in your own copy.

Lines 17-19 · `mpu_read(uint8_t reg, uint8_t *dst, size_t n)`

One call performs the standard two-part I²C register read: first *write* the register number to tell the chip where you want to read from, then, without releasing the bus, *read* the data back. Holding the bus across both halves — a repeated start rather than a stop — prevents another controller from interleaving and moving the sensor internal register pointer underneath you.

`i2c_master_transmit_receive(mpu, @, 1, dst, n, 1000)` first **writes** one byte (the register address, passed as `@` because the driver wants a buffer pointer), then issues a **repeated start** and **reads** `n` bytes into `dst`.

Holding the bus across both halves is the point: it prevents another controller from interleaving and moving the register pointer underneath you. `size_t` is the standard type for "a count of things," which is why it is used for `n` rather than `int`.

Lines 23-24 · `.clk_source / .i2c_port`

`.i2c_port = I2C_NUM_0` picks one of the chip's two hardware I²C units. `.clk_source` takes the default; as with LEDC, the choice exists because some clocks survive sleep and some do not.

Line 27 · `.glitch_ignore_cnt = 7`

Tells the hardware to ignore pulses shorter than 7 clock cycles. Free noise rejection, and a genuine help on breadboard wiring where a long jumper picks up interference. This is a hardware filter, not software — the spikes never reach your code.

Line 28 · `.flags.enable_internal_pullup = true`

A **nested field**: `flags` is itself a small struct inside the config, so you reach through it with a second dot. This switches on the chip's internal pull-ups, which return the open-drain lines HIGH when nobody is pulling them low.

Your breakout already has its own pull-ups, so enabling these too just puts two resistors in parallel and lowers the effective resistance — a non-issue at these speeds and distances, and the belt-and-braces choice for a breadboard.

Lines 30-31 · `i2c_master_bus_handle_t bus; ESP_ERROR_CHECK(i2c_new_master_bus(&bus_cfg, &bus));`

Creates the bus and fills in a bus handle. Note there are now **two** handles in play: one for the *bus* and one for the *device* on it. That separation is what lets you hang several chips off the same two wires.

Line 34 · `.dev_addr_length = I2C_ADDR_BIT_LEN_7`

Declares this device uses **7-bit** addressing, the normal case. I²C also defines a 10-bit addressing mode for buses that need more than 128 devices, which is rare.

Line 36 · `.scl_speed_hz = 400000`

400 kHz, I²C "fast mode" and the MPU's limit. Crucially this is a **per-device** setting: a slow chip sharing the same bus can be registered at 100 kHz and the driver adjusts the clock per transaction. Getting a lower rate than you expect here is a classic cause of a sampler that cannot keep up.

Line 38 · `i2c_master_bus_add_device(bus, &dev_cfg, &mpu);`

Registers the device on the bus and fills in the `mpu` handle the two helpers use. From here on, `mpu` is the sensor.

Lines 40-42 · reading WHO_AM_I

`mpu_read(0x75, &who, 1)` reads one byte from register `0x75`. This is **step five of professional board bring-up**, not ceremony: every I²C chip has an ID register, and confirming it proves your wiring, your address and your bus timing before you trust a single data byte.

`0x%02X` formats the byte as exactly two uppercase hex digits with a leading zero — `%X` for uppercase hex, `02` for minimum width two, zero-padded. Without the `02` a value of `0x07` would print as just `7`.

Your part answers `0x70`. See the correction box in §5.1: that is an MPU-6500 identifying itself correctly, not a fault.

Line 44 · `mpu_write(0x6B, 0x00);`

Writes zero to **PWR_MGMT_1**. On an MPU-6050 this clears a SLEEP bit that is set at power-on — the part boots asleep — and on any of these parts writing zero leaves it awake and internally clocked. Omitting this on a 6050 is the classic "all my readings are exactly zero" bug. Trace this exact line back to its register-map entry and confirm the bit for yourself.

Line 45 · `vTaskDelay(pdMS_TO_TICKS(50));`

Gives the sensor time to stabilise after waking. Datasheets specify start-up times, and ignoring them produces intermittent garbage on the first few reads — the kind of bug that disappears when you add a print statement and comes back when you remove it.

Line 49 · `mpu_read(0x3B, r, 6);`

Reads six consecutive bytes starting at **ACCEL_XOUT_H**. Six because three axes × two bytes each, and the sensor lays them out consecutively so one transaction gets all of them. Doing three separate two-byte reads would work but would be three times the bus traffic and could straddle a sensor update, giving you X from one sample and Z from the next.

Lines 50-52 · `int16_t ax = (int16_t)((r[0] << 8) | r[1]);`

The line that rewards the most attention in this lab. Three things happen.

The shift and OR. The sensor sends a high byte and a low byte separately. `r[0] << 8` moves the high byte into the upper half of a 16-bit value; `| r[1]` fills the lower half. That is **big-endian** reassembly — most-significant byte first.

Integer promotion, which makes it safe. `r[0]` is a `uint8_t`, and C automatically widens it to a 32-bit `int` before the shift. So nothing falls off the top: the intermediate result has plenty of room.

The `(int16_t)` cast, which is the part that matters. Acceleration can be negative, and `int16_t` is a *signed* 16-bit type, so the cast makes the top bit act as a sign bit in two's complement. Without it, a value like `0xC000` would read as 49152 instead of −16384, and a tilted or inverted board would report a huge positive number. **TM4C bridge:** this is exactly what you did after an ADC read — hardware hands you bytes, you reconstruct the number and interpret its sign.

Lines 54-55 · `ax / 16384.0f`

Converts raw counts to **g**. At the default ±2g full-scale range the sensor produces 16384 counts per g, a number that comes from the sensitivity table in the product specification — not folklore. Change the range via **ACCEL_CONFIG** (`0x1C`) and this divisor changes with it.

The `.0f` matters: it makes the literal a `float`, which forces floating-point division. Writing `ax / 16384` would be integer division and would give you 0 or 1 with everything after the decimal point discarded — a silent, complete loss of precision.

`%.2f` is three things at once: `+` always shows the sign so positive and negative values line up in columns, `.2` gives two decimal places, `f` is a float.

Line 56 · `vTaskDelay(pdMS_TO_TICKS(200));`

Five readings a second — fast enough to see the response when you tilt the board, slow enough to read in the monitor. Lab 7 replaces this with a hardware timer at 1 kHz, and §2.3 explains why a task delay could never get there.

5.6 Docs & references

DOCS

Trace every line to its source

`i2c_master_bus_config_t`, `i2c_new_master_bus`, `i2c_device_config_t`, `i2c_master_bus_add_device`, `i2c_master_transmit`, `i2c_master_transmit_receive` — ESP-IDF API Reference → Peripherals → **I²C**. Make sure you are reading the **i2c_master** driver, not the deprecated legacy `i2c.h` API — a great deal of code on the internet still uses the old one and will not compile against yours.

MPU-6500 Register Map and Descriptions — source of `0x75`, `0x6B`, `0x3B`, `0x1C`. Search TDK InvenSense for "MPU-6500 register map."

MPU-6500 Product Specification — source of `16384` LSB/g. Find the accelerometer sensitivity table and see the number in context, along with the values for $\pm 4g$, $\pm 8g$ and $\pm 16g$.

I²C specification — NXP UM10204 if you want the authoritative account of START/STOP, arbitration and clock stretching.

5.7 Verify, troubleshoot, your turn

Verify. `WHO_AM_I` prints `0x70` on your part. Then, with the board flat and still, `az` reads about **+1.00 g** and the other two sit near zero — that is gravity, and it is the physical sanity check that proves the whole chain from bus to units. Tilt the board and watch the g redistribute across axes; invert it and `az` goes to about -1.00 . A magnitude of 0.98 to 1.00 is normal; these are cheap parts with a small offset.

On the analyzer, the decoder should show address `0x68` followed by the register byte and the data. Measure SCL with cursors and you should find about 399 kHz, confirming your `400000` on the actual wire.

Troubleshoot

Symptom	Cause and fix
<code>WHO_AM_I</code> reads <code>0x00</code> or the transaction NAKs	SDA/SCL swapped — the most likely cause by far. Check by label.
<code>WHO_AM_I</code> reads <code>0x70</code>	Not a fault. That is your MPU-6500 identifying itself correctly.

Symptom	Cause and fix
All readings are exactly zero	The wake write to <code>0x6B</code> did not happen or failed.
Readings look wildly large or the sign flips oddly	Missing the <code>(int16_t)</code> cast.
Sporadic NAKs	Long jumper wires, or the sensor is on 5V. Shorten the leads; confirm 3V3.
Analyzer decode is empty but traces look fine	SDA and SCL assigned the wrong way round in the decoder. A settings fix, not a rewiring job.

Your turn

1. **Change the full-scale range.** Write to `ACCEL_CONFIG (0x1C)` to select $\pm 4g$, then work out from the product specification what the new divisor must be, and confirm that gravity still reads 1.00 g. This is the exercise that proves you can use a register map rather than copy constants.
2. **Read the gyroscope too.** Its registers start at `0x43` and follow the same big-endian signed layout. Find its sensitivity in the datasheet yourself.
3. **Capture and annotate one full transaction** on the analyzer. Identify START, the address with its R/W bit, each ACK, the register byte, the data, and STOP. Save the screenshot into `debug_log.md`. Being able to point at a real capture and narrate it is a genuinely strong interview moment.

Errata and corrections

Defects confirmed against the hardware and the vendor documentation, with the evidence that established each one. They are printed rather than quietly patched because how an error was caught is usually more instructive than the error.

Additional corrections

Unchecked returns. The canonical listing ignores the `esp_err_t` from `mpu_read` and `mpu_write`. A failed read leaves `r[6]` holding stale or uninitialised bytes, which are then parsed as acceleration. Check every bus operation and distinguish NACK, timeout and a bad WHO_AM_I in your logs.

The series at a glance

Each lab is a complete, independently buildable ESP-IDF project. This is Lab 5.

#	Lab	Phase
00	Toolchain: hello, ESP32-S3	Foundations
01	GPIO: RGB LED and BOOT button	Foundations
02	FreeRTOS tasks and drift-free timing	Foundations
03	UART and the loopback test	Foundations
04	Hardware timers and the 1 kHz problem	Foundations
05 ←	I²C and the IMU bring-up	Foundations
06	PWM with the LEDC peripheral	Foundations
07	The sensor pipeline (capstone)	Foundations
08	GPIO interrupts and ISR-safe code	Past the coursework
09	SPI, full-duplex and fast	Past the coursework
10	NVS: data that survives power loss	Past the coursework
11	Unit testing with Unity	Past the coursework
12	Deep sleep and waking up	Past the coursework
13	Wi-Fi station and JSON telemetry	Past the coursework
14	BLE GATT with NimBLE	Past the coursework
15	OTA and the rollback safety net	Past the coursework
16	Priority inversion and mutexes	Past the coursework
17	CAN (TWAI): broadcast by message ID	Past the coursework
18	PID, anti-windup and derivative kick	Past the coursework
19	The ML inference harness	Gap-closers
20	Continuous ADC: DMA-backed acquisition	Gap-closers
21	On-chip debugging: JTAG, OpenOCD, GDB	Gap-closers
22	A real display (SPI TFT + DMA)	Gap-closers
23	Production test mode	Gap-closers

Written against ESP-IDF v5.5 on an ESP32-S3 N16R8 module. Pin assignments are choices, not constraints — the ESP32-S3 GPIO matrix routes almost any peripheral to almost any pin, with the exception of GPIO26–GPIO37, which are committed to the on-board flash and PSRAM and must never be used.